



Números (pseudo)aleatorios y generación de variables aleatorias

Práctica No. 2

S. Barrera Meza¹

1. Cod: 160005004, Ing. Sistemas.

Facultad de Ciencias Básicas e Ingenierías. Programa de Ingeniería de Sistemas.

Fecha: Septiembre de 2026

Resumen

Se implementaron y compararon cuatro métodos de generación de números pseudoaleatorios (MidSquare, congruencial mixto, Lagged-Fibonacci y Shift-Register), midiendo para cada secuencia de 100 números el tamaño del ciclo y aplicando una prueba de uniformidad (chi-cuadrado) y una de aleatoriedad (rachas respecto a la media). Los generadores MidSquare y el congruencial con $m = 200$ resultaron los peores por sus ciclos cortos y sus estados absorbentes, mientras que los congruenciales con $m = 2^{31} - 1$ y $m = 2^{48}$ y el Shift-Register pasaron ambas pruebas. Con esos números se estimaron por Monte Carlo las siete integrales de los ejercicios 3 a 9 del capítulo 3 de [1], se generaron variables aleatorias discretas por transformada inversa y por composición, y se repitió la simulación de la fila del banco del Laboratorio 1 usando generadores propios de Poisson y Binomial.

Palabras clave — números pseudoaleatorios, periodo, pruebas estadísticas, transformada inversa, Monte Carlo.

1. Objetivos

- Generar secuencias de números pseudoaleatorios con los métodos de MidSquare, congruencial mixto y otros métodos del artículo [2].
- Calcular el tamaño del ciclo de cada secuencia y evaluar si cumple los criterios de uniformidad y aleatoriedad.
- Usar números aleatorios para evaluar integrales por el método de Monte Carlo.
- Generar variables aleatorias discretas por el método de la transformada inversa y por el método de composición.
- Repetir la simulación ad hoc del Laboratorio 1 con generadores propios de Poisson y Binomial.

2. Consulta previa

Variable aleatoria. Función que le asigna un número real a cada resultado de un experimento aleatorio, como la suma de dos dados.



Distribución de probabilidad. Regla que indica con qué probabilidad la variable toma cada valor: función de masa $p(x) = P\{X = x\}$ en el caso discreto y función de densidad $f(x)$ en el continuo.

Discreta frente a continua. La discreta se define sobre un conjunto contable de valores y ahí sí tiene sentido $P\{X = x\}$; la continua se define sobre un intervalo, la probabilidad de un punto es cero y solo se pregunta por intervalos.

Uniforme $U(0,1)$. $f(x) = 1$ para $0 < x < 1$, $F(x) = x$, media $1/2$ y varianza $1/12$. Es la distribución que imitan los generadores de números pseudoaleatorios.

Poisson. $P\{X = i\} = e^{-\lambda} \lambda^i / i!$ para $i = 0, 1, 2, \dots$, con media y varianza iguales a λ . Cuenta eventos en un intervalo de tiempo o espacio.

Binomial. $P\{X = i\} = \binom{n}{i} p^i (1-p)^{n-i}$; cuenta los éxitos en n ensayos independientes, con media np y varianza $np(1-p)$.

Propiedad i.i.d. Independiente e idénticamente distribuida: todos los valores siguen la misma distribución y ninguno depende de los anteriores. Por eso a un generador se le aplican pruebas de uniformidad y de aleatoriedad.

Prueba estadística. Procedimiento para decidir si se rechaza una hipótesis nula H_0 : se calcula un estadístico, se compara con un valor crítico para un nivel de significancia α y se rechaza H_0 si cae en la región de rechazo. No rechazar no demuestra H_0 .

3. Generación de números pseudoaleatorios

Se implementaron cuatro generadores en Python: MidSquare (elevar la semilla de d dígitos al cuadrado y tomar los d dígitos centrales), el congruencial mixto $X_{n+1} = (aX_n + c) \bmod m$, el Lagged-Fibonacci $X_n = (X_{n-17} + X_{n-5}) \bmod 2^{31}$ y el Shift-Register (xorshift de 32 bits con desplazamientos 13, 17 y 5). Con cada juego de parámetros se generaron $n = 100$ números. La Tabla 1 muestra cuántos valores distintos produjo cada secuencia, que ya da una primera señal de su calidad.

Tabla 1: Valores distintos dentro de los 100 números generados por cada secuencia.

Método	Parámetros	Valores distintos
MidSquare	semilla 1931	7
MidSquare	semilla 675248	100
MidSquare	semilla 4567	22
MidSquare	semilla 12122	88
MidSquare	semilla 7179345	100
Congruencial	$X_0 = 127, a = 115, c = 0, m = 128$	32
Congruencial	$X_0 = 43, a = 25, c = 5, m = 48$	48
Congruencial	$X_0 = 13, a = 16807, c = 0, m = 2^{31} - 1$	100
Congruencial	$X_0 = 29022024, a = 25214903917, c = 11, m = 2^{48}$	100
Congruencial	$X_0 = 3, a = 5, c = 7, m = 200$	9
Lagged-Fibonacci	$r = 17, s = 5, m = 2^{31}$, semilla 160005004	100
Shift-Register	32 bits, semilla 160005004	100

3.1. Tamaño de ciclo y pruebas estadísticas

El tamaño del ciclo se obtuvo guardando los estados ya visitados y midiendo la distancia entre las dos apariciones del primer estado repetido, con un límite de búsqueda de 300000 iteraciones. El transitorio es la cantidad de valores que salen antes de entrar al ciclo.



La prueba de uniformidad fue la de chi-cuadrado con $k = 10$ clases de igual ancho sobre $[0, 1)$. Con H_0 : los números provienen de una $U(0, 1)$,

$$\chi_0^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}, \quad E_i = \frac{n}{k} = 10, \quad (1)$$

y para $\alpha = 0,05$ con 9 grados de libertad el valor crítico es 16.919.

La prueba de aleatoriedad fue la de rachas respecto a la media: se marca cada número según quede por encima o por debajo de la media, se cuenta el número de rachas b y se compara

$$Z_0 = \frac{b - \mu_b}{\sigma_b}, \quad \mu_b = \frac{2n_1n_2}{n} + \frac{1}{2}, \quad \sigma_b = \sqrt{\frac{2n_1n_2(2n_1n_2 - n)}{n^2(n-1)}}, \quad (2)$$

con el valor crítico 1.96 para $\alpha = 0,05$. Los resultados están en la Tabla 2.

Tabla 2: Tamaño de ciclo y resultado de las pruebas de uniformidad y aleatoriedad.

Generador	Transitorio	Ciclo	χ_0^2	¿Uniforme?	Z_0	¿Aleatorio?
MidSquare 1931	7	1	860.60	NO	-7,117	NO
MidSquare 675248	210	1	12.40	SI	1.709	SI
MidSquare 4567	19	4	109.40	NO	-0,302	SI
MidSquare 12122	88	1	52.20	NO	-3,306	NO
MidSquare 7179345	848	20	15.00	SI	-1,048	SI
GLC $m = 128$	0	32	6.80	SI	-0,905	SI
GLC $m = 48$	0	48	1.20	SI	-0,905	SI
GLC $m = 2^{31} - 1$	-	> 300000	12.00	SI	0.541	SI
GLC $m = 2^{48}$	-	> 300000	4.80	SI	0.318	SI
GLC $m = 200$	2	8	22.80	NO	-4,923	NO
Lagged-Fibonacci	-	> 300000	6.40	SI	1.987	NO
Shift-Register	-	> 300000	5.00	SI	0.909	SI

Los periodos medidos coinciden con lo que predice el teorema de Hull-Dobell, según el cual un congruencial mixto tiene periodo completo m si c y m son primos relativos, $a - 1$ es divisible por todos los factores primos de m y, cuando m es divisible por 4, $a - 1$ también lo es. El caso $m = 48$ cumple las tres condiciones y por eso su periodo es 48; el caso $m = 200$ falla la segunda, porque $a - 1 = 4$ no es divisible por 5, y por eso su periodo se cae a 8. El caso $m = 128$ es multiplicativo ($c = 0$) y para $m = 2^k$ el máximo alcanzable es $m/4 = 32$, que fue lo obtenido. Los casos $m = 2^{31} - 1$ y $m = 2^{48}$ tienen periodos de $2^{31} - 2$ y 2^{48} , imposibles de recorrer por fuerza bruta.

3.2. ¿Cuáles son los peores generadores y por qué?

Los peores son los MidSquare. Con la semilla 1931 la secuencia cae en cero a los 7 pasos y los 93 números restantes son todos 0, de manera que falla las dos pruebas por amplio margen ($\chi_0^2 = 860,60$ y $Z_0 = -7,117$). Con la semilla 12122 ocurre lo mismo a los 88 pasos y con 4567 el ciclo es de apenas 4 valores. El caso más engañoso es la semilla 675248: los primeros 100 números pasan ambas pruebas, pero en el paso 210 la secuencia llega al número 625000, que es un punto fijo, y a partir de ahí repite ese valor indefinidamente. Ese es el riesgo real del método, que puede verse bien en una muestra corta y estar dañado más adelante, sin que exista un criterio para escoger la semilla de antemano.



El otro generador malo es el congruencial con $m = 200$, que no cumple Hull-Dobell, tiene periodo 8 y rechaza tanto uniformidad como aleatoriedad. En todos estos casos el problema de fondo es el mismo: ciclos demasiado cortos o estados absorbentes que hacen que los valores se repitan y dejen de ser independientes.

3.3. ¿Cuáles son los mejores generadores y por qué?

Los mejores son los congruenciales con $m = 2^{31} - 1$ y $m = 2^{48}$ y el Shift-Register. Los tres pasan las dos pruebas con holgura (χ_0^2 de 12.00, 4.80 y 5.00 frente al crítico 16.919, y $|Z_0|$ de 0.541, 0.318 y 0.909 frente a 1.96) y tienen periodos de $2^{31} - 2$, 2^{48} y $2^{32} - 1$, de modo que no se repiten en ninguna simulación de tamaño razonable. Ninguno de los tres muestra estructura en los pares consecutivos (u_i, u_{i+1}) , a diferencia del congruencial con $m = 48$, cuyos valores solo pueden caer en 48 posiciones. El Lagged-Fibonacci pasa la prueba de uniformidad y tiene un periodo teórico del orden de 10^{14} , pero su $Z_0 = 1.987$ queda apenas por encima del valor crítico, lo que concuerda con la correlación que arrastra el método por sumar valores anteriores.

Conviene resaltar que los congruenciales con $m = 128$ y $m = 48$ pasaron las dos pruebas con 100 números y aun así no sirven, porque su secuencia se repite cada 32 y 48 valores. Pasar las pruebas estadísticas no basta si el periodo es corto: hay que revisar las dos cosas.

4. Uso de números aleatorios para evaluar integrales

Si $U \sim U(0, 1)$, entonces $\int_0^1 g(x)dx = E[g(U)]$, que se estima con el promedio de $g(U_1), \dots, g(U_k)$. Cuando los límites no son 0 y 1 se hace un cambio de variable: $x = a + (b - a)y$ para el intervalo $[a, b]$, y $x = 1/y - 1$, con $dx = dy/y^2$, para el intervalo $[0, \infty)$. Los ejercicios 3 a 9 se resolvieron con $k = 100000$ repeticiones, tomando los números uniformes del generador congruencial con $m = 2^{48}$ y no de la librería `random`. El valor exacto se calculó con la función `quad` de `scipy` para comparar.

Tabla 3: Estimación por Monte Carlo de las integrales de los ejercicios 3 a 9.

Ejercicio	Monte Carlo	Error est.	Exacto	Diferencia
3. $\int_0^1 e^x dx$	6.29411	0.01038	6.31656	0.02246
4. $\int_0^1 (1 - x^2)^{3/2} dx$	0.59106	0.00105	0.58905	0.00202
5. $\int_{-2}^2 e^{x+x^2} dx$	91.52547	0.76420	93.16275	1.63728
6. $\int_0^\infty x(1+x^2)^{-2} dx$	0.50039	0.00106	0.50000	0.00039
7. $\int_{-\infty}^\infty e^{-x^2} dx$	1.76822	0.00447	1.77245	0.00423
8. $\int_0^1 \int_0^1 e^{(x+y)^2} dy dx$	4.90750	0.01906	4.89916	0.00834
9. $\int_0^\infty \int_0^x e^{-(x+y)} dy dx$	0.50080	0.00231	0.50000	0.00080

Las siete estimaciones quedan dentro de dos o tres errores estándar del valor exacto: el ejercicio 4 da 0.59106 frente a $3\pi/16 = 0.58905$, el 7 da 1.76822 frente a $\sqrt{\pi} = 1.77245$ y los ejercicios 6 y 9 dan prácticamente 0.5. El único que se aleja en valor absoluto es el ejercicio 5, pero la integral vale 93 y su error estándar es 0.76, es decir un error relativo del 1.8 %, que se explica porque e^{x+x^2} crece muchísimo cerca de $x = 2$ y la varianza de $g(U)$ es grande. En los ejercicios 6, 7 y 9, definidos sobre dominios infinitos, lo que hizo posible la estimación fue el cambio $x = 1/u - 1$, que lleva todo el dominio al intervalo $(0, 1)$ donde el generador produce los números; en el ejercicio 9 se usó además la función indicadora $I(y < x)$ sugerida en el libro para extender la integral interna hasta infinito.



5. Generación de variables aleatorias discretas

Con el método de la transformada inversa se toma $X = j$ cuando U cae entre la acumulada hasta $j - 1$ y la acumulada hasta j . Todos los puntos de esta sección usan el congruencial con $X_0 = 2391$, $a = 25214903917$, $c = 11$ y $m = 2^{48}$.

Tabla 4: Frecuencias relativas de los 100 valores generados en los puntos a) y b).

j	Teórica a)	Simulada a)	Teórica b)	Simulada b)
1	0.20	0.21	0.30	0.32
2	0.15	0.14	0.20	0.22
3	0.25	0.27	0.35	0.29
4	0.40	0.38	0.15	0.17

En los dos casos las frecuencias relativas quedan cerca de las teóricas, con una desviación máxima de 0.06. En el punto c), con $p_1 = 1/3$ y $p_2 = 2/3$, la proporción de valores iguales a 1 fue 0.3400 con $n = 100$, 0.3280 con $n = 1000$ y 0.3316 con $n = 10000$, cada vez más cerca de $1/3$. La diferencia no baja de forma monótona porque el error es aleatorio, pero su tamaño típico sí baja como $\sqrt{p(1-p)/n}$, que pasa de 0.047 a 0.0047 entre esos dos extremos.

5.1. Método de composición (punto d)

Los cinco primeros valores tienen la misma probabilidad 0.06 y $0,06 = 0,6 \times 0,1$, así que la distribución se puede escribir como la mezcla

$$p_j = 0,6 p_j^{(1)} + 0,4 p_j^{(2)}, \quad (3)$$

donde $p^{(1)}$ es la uniforme discreta sobre $1, \dots, 10$ y $p^{(2)} = (p_j - 0,06)/0,4$ queda concentrada en $6, \dots, 10$ con probabilidades 0.225, 0.175, 0.20, 0.225 y 0.175. El algoritmo genera U_1 : si $U_1 < 0,6$ devuelve $\lfloor 10U_2 \rfloor + 1$, y en caso contrario genera el valor a partir de $p^{(2)}$ por transformada inversa. Así, el 60 % de las veces basta con una multiplicación y un truncado, sin recorrer la tabla acumulada.

Tabla 5: Método de composición: proporciones obtenidas con 100 y con 10000 valores.

j	1	2	3	4	5	6	7	8	9	10
Teórica	0.06	0.06	0.06	0.06	0.06	0.15	0.13	0.14	0.15	0.13
$n = 100$	0.09	0.06	0.00	0.12	0.09	0.15	0.17	0.13	0.07	0.12
$n = 10000$	0.058	0.063	0.059	0.056	0.062	0.153	0.127	0.140	0.156	0.127

Con 100 valores las proporciones se alejan bastante (el valor 3 no salió ninguna vez), lo cual es normal porque se esperan apenas 6 apariciones de cada uno de los primeros cinco valores; con 10000 valores ya coinciden con las teóricas hasta la segunda cifra decimal, lo que confirma que el algoritmo está bien implementado.

6. Simulación ad hoc con variables aleatorias discretas

Se repitieron las secciones 5.1 y 5.2 del Laboratorio 1 con 20 clientes, pero generando los tiempos entre llegadas con un generador propio de Poisson con $\lambda = 10$ y los tiempos de servicio



con uno Binomial con $n = 10$ y $p = 0,40$, ambos por transformada inversa con las recursiones

$$p_{i+1} = \frac{\lambda}{i+1} p_i \quad (p_0 = e^{-\lambda}), \quad p_{i+1} = \frac{n-i}{i+1} \cdot \frac{p}{1-p} p_i \quad (p_0 = (1-p)^n). \quad (4)$$

Antes de simular se comprobaron los generadores con 10000 valores: la Poisson dio media 10.0173 y varianza 9.8798 (teóricas 10 y 10) y la Binomial media 3.9986 y varianza 2.3594 (teóricas 4 y 2.4), de modo que las dos implementaciones reproducen la distribución esperada.

Tabla 6: Simulación de la fila del banco con 20 clientes. Tiempos en minutos.

Cliente	T entre llegadas	T de llegada	T de servicio	Inicia servicio	Espera en cola	Termina servicio	En el sistema	Ocio del servidor
1	0	0	4	0	0	4	4	0
2	12	12	7	12	0	19	7	8
3	11	23	3	23	0	26	3	4
4	12	35	4	35	0	39	4	9
5	12	47	2	47	0	49	2	8
6	9	56	6	56	0	62	6	7
7	11	67	4	67	0	71	4	5
8	14	81	1	81	0	82	1	10
9	9	90	3	90	0	93	3	8
10	12	102	6	102	0	108	6	9
11	8	110	3	110	0	113	3	2
12	4	114	7	114	0	121	7	1
13	12	126	2	126	0	128	2	5
14	12	138	7	138	0	145	7	10
15	6	144	4	145	1	149	5	0
16	7	151	2	151	0	153	2	2
17	7	158	2	158	0	160	2	5
18	6	164	5	164	0	169	5	4
19	7	171	2	171	0	173	2	2
20	9	180	5	180	0	185	5	7
Total	180		79		1		80	106

Tabla 7: Medidas de desempeño comparadas con la primera corrida del Laboratorio 1.

Medida de desempeño	Laboratorio 1	Laboratorio 2
Average time in system (min)	3.5500	4.0000
Percent idle time	0.3981	0.5730
Average waiting time per customer (min)	0.4500	0.0500
Fraction having to wait	0.3000	0.0500
Average waiting time of those who waited (min)	1.5000	1.0000

6.1. Análisis de los resultados frente al Laboratorio 1

De los 20 clientes solo uno tuvo que esperar (el cliente 15, un minuto), la espera promedio fue de 0.05 minutos y el servidor estuvo desocupado el 57.3 % del tiempo. El tiempo promedio en el sistema fue de 4 minutos, casi igual al tiempo promedio de servicio ($79/20 = 3,95$), justamente porque casi nadie hizo fila.

La diferencia principal está en la carga del sistema. En el Laboratorio 1 los tiempos entre llegadas eran uniformes discretos entre 1 y 10 (media 5.5) y los de servicio uniformes entre 1 y 6 (media 3.5), lo que da una utilización aproximada de $3,5/5,5 = 0,64$; ahora las llegadas son Poisson con $\lambda = 10$ y los servicios Binomial con media 4, es decir $4/10 = 0,40$. Al llegar los clientes más espaciados el sistema se descongestiona: la fracción de clientes que esperan baja



de 0.30 a 0.05, la espera promedio pasa de 0.45 a 0.05 minutos y el tiempo ocioso del servidor sube de 39.8 % a 57.3 %. Otra diferencia está en la forma de las distribuciones: la uniforme reparte la probabilidad por igual entre todos sus valores, mientras que la Poisson y la Binomial se concentran alrededor de su media, de modo que no aparecieron valores extremos.

Las similitudes están en la mecánica de la simulación, ya que la tabla, las fórmulas de cada columna y las medidas de desempeño son las mismas y lo único que cambió fue el origen de los dos tiempos aleatorios. Se mantiene también el comportamiento general de que, mientras el tiempo medio de servicio sea menor que el tiempo medio entre llegadas, la cola no crece. La ventaja frente al Laboratorio 1 es que ya no se necesitan dados ni ruletas: la simulación es repetible con la misma semilla y puede escalarse a miles de clientes sin trabajo adicional. Eso sí, una sola corrida de 20 clientes no permite concluir demasiado, como se vio en el Laboratorio 1 con las diez réplicas y los intervalos de confianza; aquí la espera promedio de los que esperaron está calculada con un solo cliente.

7. Conclusiones

- El método de MidSquare no es utilizable en la práctica: según la semilla se degrada hacia el cero o hacia un punto fijo, y no existe un criterio previo para saber qué semilla se comportará bien.
- El generador congruencial mixto sí es utilizable, pero solo con parámetros bien escogidos. El teorema de Hull-Dobell explicó exactamente los periodos medidos, y en el caso $m = 200$ bastó con que fallara una condición para que el periodo cayera a 8.
- Pasar las pruebas de uniformidad y aleatoriedad no es suficiente por sí solo: los congruenciales con $m = 48$ y $m = 128$ las pasaron con 100 números y aun así son malos generadores por su periodo corto.
- El método de Monte Carlo estimó las siete integrales con buena precisión, incluidas las de dominio infinito gracias al cambio de variable $x = 1/u - 1$, y su error decrece como $1/\sqrt{k}$, de modo que ganar precisión resulta costoso en cómputo.
- Con los mismos números uniformes se generaron las variables discretas por transformada inversa y por composición y se alimentó la simulación de la fila del banco, de manera que un generador defectuoso habría contaminado todos los resultados del laboratorio.

Referencias

- [1] S. M. Ross, *Simulación*, 2nd ed. Pearson Press, 1999.
- [2] A. M. M. Herrera, "Números aleatorios. historia, teoría y aplicaciones," *Ingeniería y Desarrollo*, no. 8, pp. 49–69, 2000.